

Homework 7

Sync API

Endpoint API

```
func (api *OrderAPI) ProcessOrderSync(c *gin.Context) {  
    // Parse request body  
    ***  
  
    makePayment()  
  
    // Update status to completed  
    order.Status = "completed"  
    api.store.SetOrder(order.OrderID, &order)  
  
    // Return success response  
    response := OrderResponse{  
        OrderID:    order.OrderID,  
        Status:     order.Status,  
        Message:    "Order processed successfully",  
        Timestamp:  time.Now().Format(time.RFC3339),  
    }  
  
    c.JSON(http.StatusOK, response)  
}
```

```
func makePayment() {  
    paymentSemaphore <- struct{}{}  
    defer func() {  
        <-paymentSemaphore  
    }()  
    time.Sleep(3 * time.Second)  
}
```

VPC Setting

```
resource "aws_vpc" "main" {
  cidr_block      = "10.0.0.0/16"
  enable_dns_hostnames = true
  enable_dns_support  = true

  tags = {
    Name = "${var.service_name}-vpc"
  }
}

# Create public subnets
resource "aws_subnet" "public" {
  count                = 2
  vpc_id              = aws_vpc.main.id
  cidr_block          = "10.0.${count.index + 1}.0/24"
  availability_zone    =
data.aws_availability_zones.available.names[count.index]
  map_public_ip_on_launch = true

  tags = {
    Name = "${var.service_name}-public-subnet-${count.index + 1}"
    Type = "public"
  }
}

# Create private subnets
resource "aws_subnet" "private" {
  count                = 2
  vpc_id              = aws_vpc.main.id
  cidr_block          = "10.0.${count.index + 10}.0/24"
  availability_zone    =
data.aws_availability_zones.available.names[count.index]

  tags = {
    Name = "${var.service_name}-private-subnet-${count.index + 1}"
    Type = "private"
  }
}
```

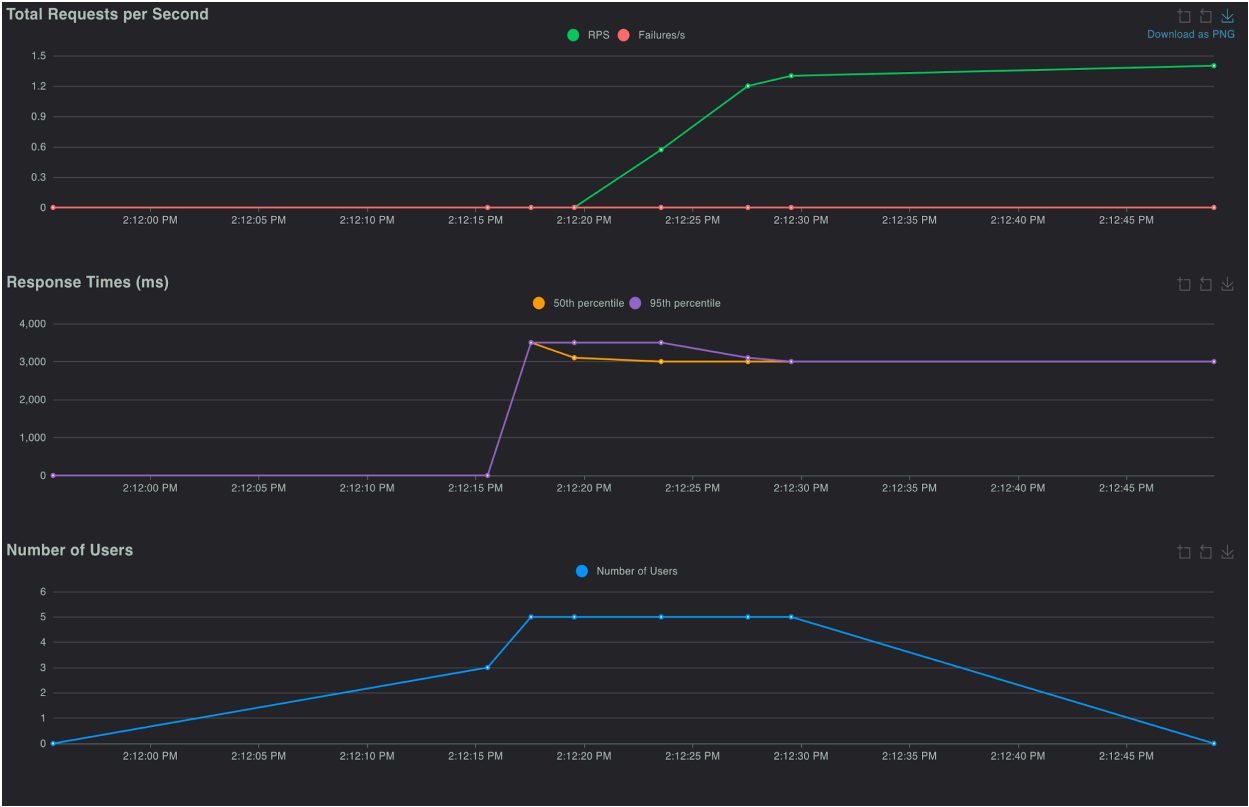
ALB Setting

```
# Target Group for Fargate tasks
resource "aws_lb_target_group" "this" {
  name           = "${var.service_name}-tg"
  port           = var.container_port
  protocol       = "HTTP"
  vpc_id         = var.vpc_id
  target_type    = "ip" # Required for Fargate
  deregistration_delay = 30 # Reduce from default 300s to 30s

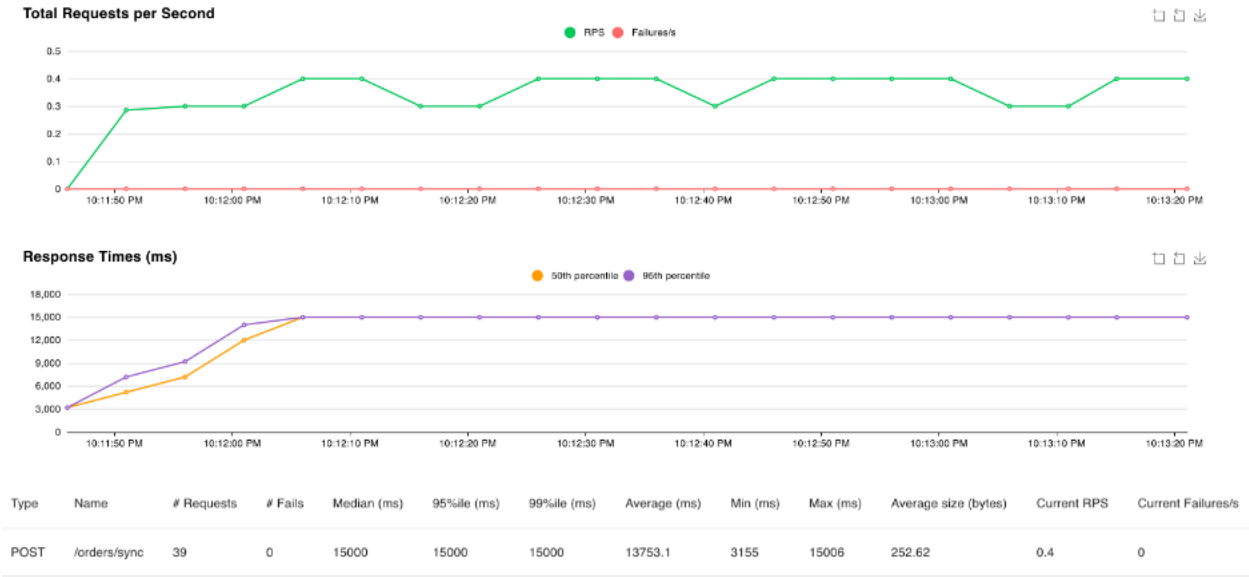
  health_check {
    enabled            = true
    healthy_threshold = 2
    interval           = 30
    matcher            = "200"
    path               = var.health_check_path
    port               = "traffic-port"
    protocol           = "HTTP"
    timeout            = 5
    unhealthy_threshold = 2
  }
}
```

Normal Operation

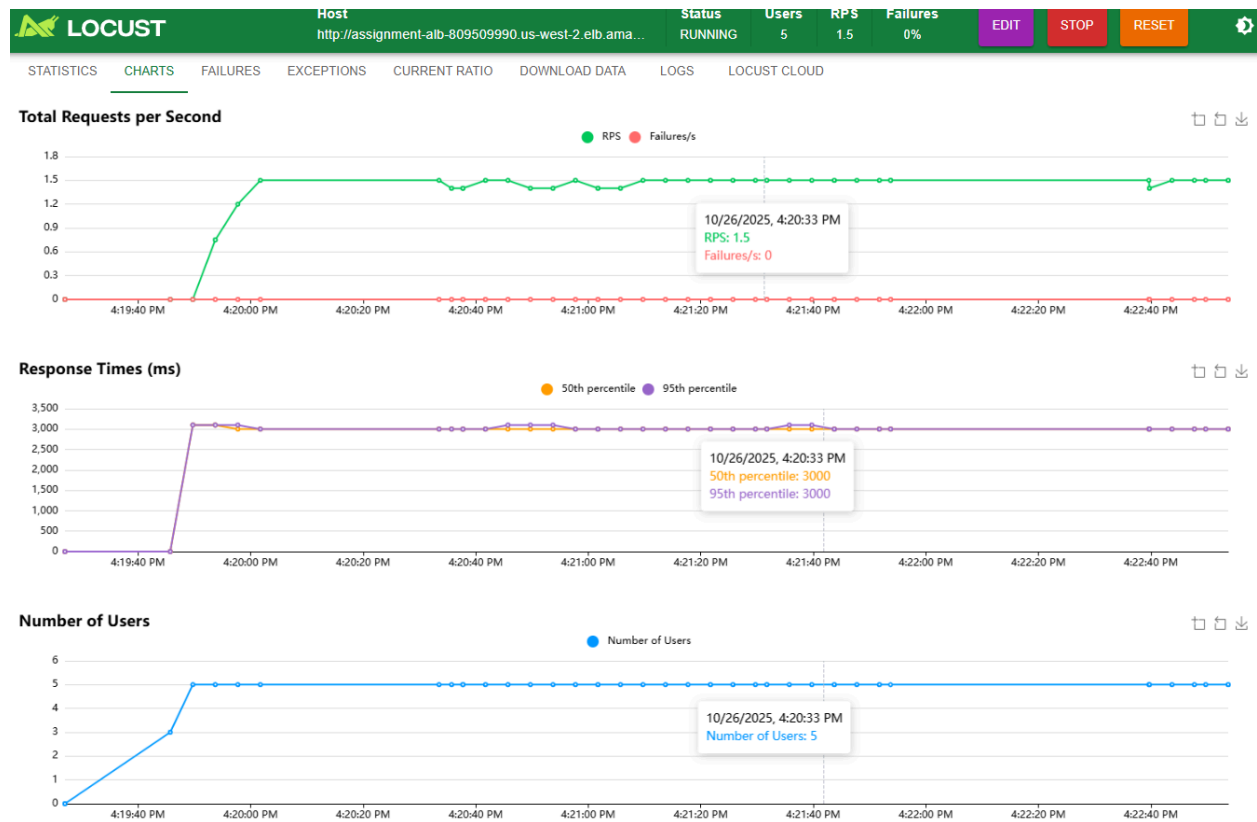
wenshuang



Zhixiao:

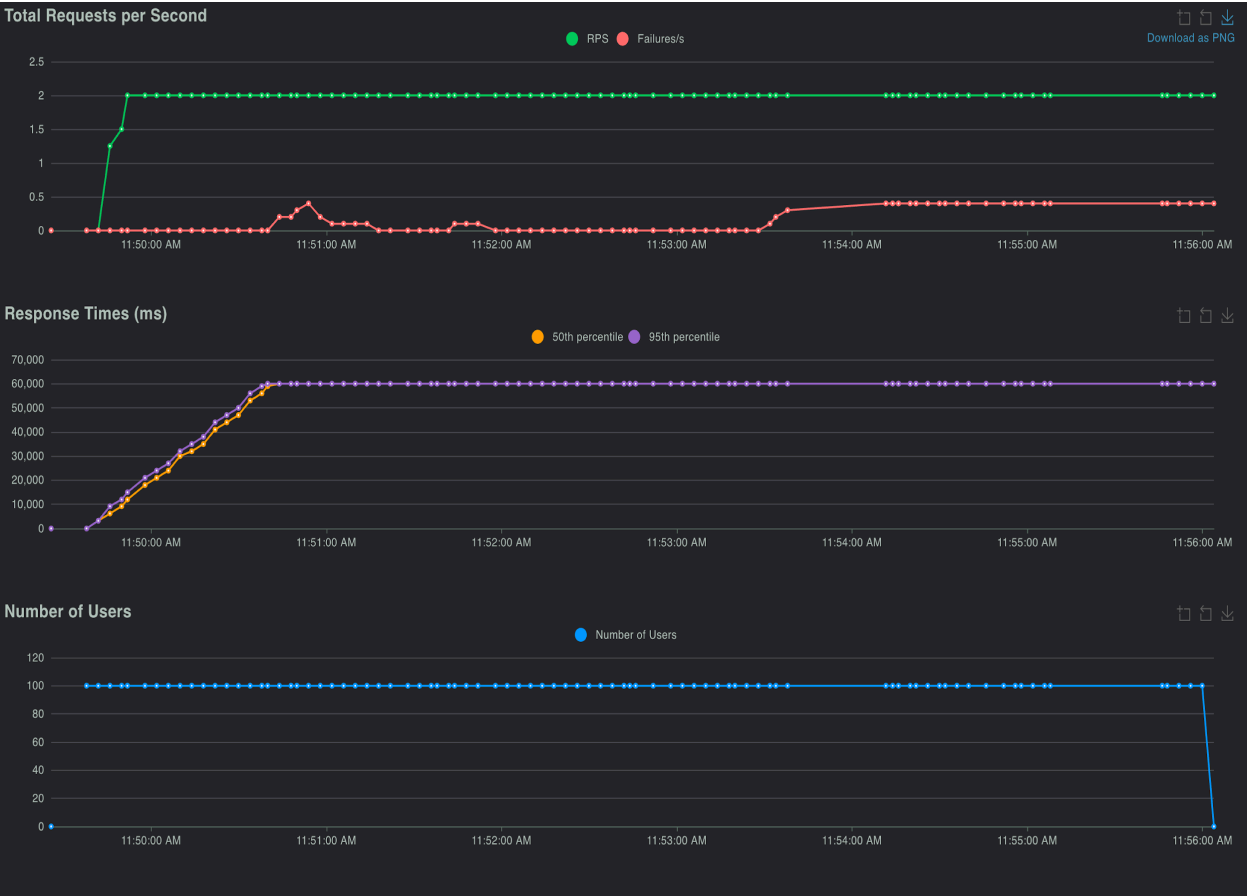


Snahil

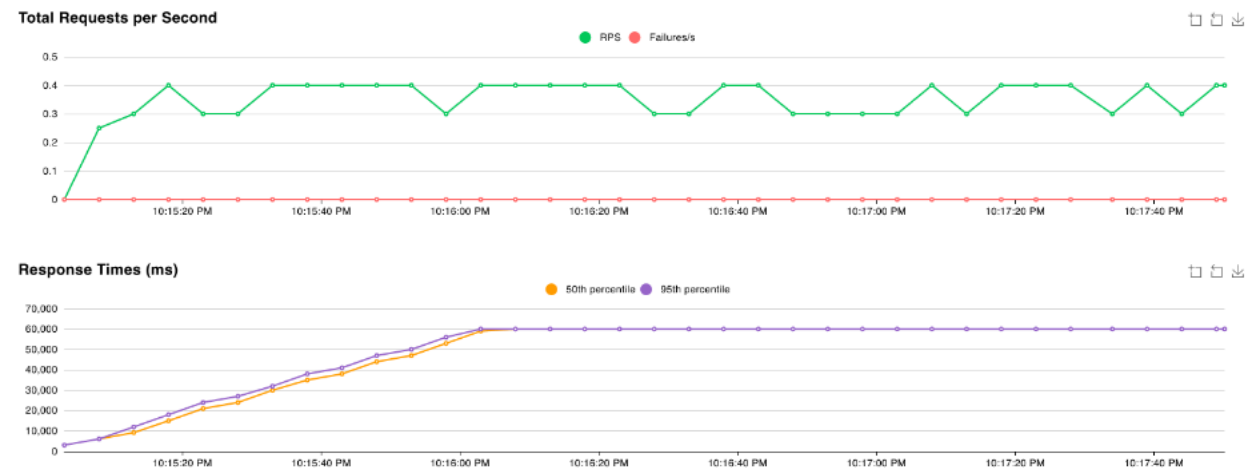


Flash Sale

wenshuang

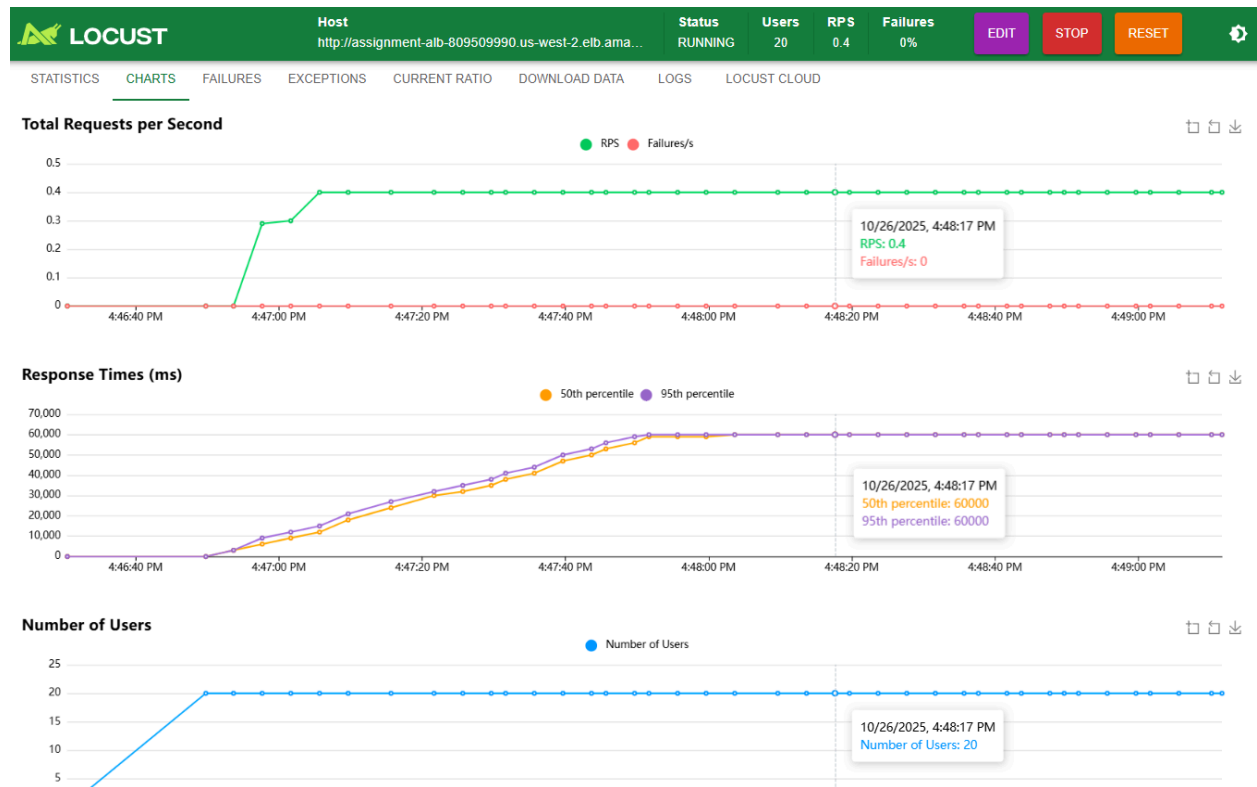


Zhixiao:



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/orders/sync	60	0	59978.47	60000	60000	50204.39	3106	59978	257.98	0.4	0

SNahil



Async with SNS/SQS

SNS/SQS

```
# SNS Topic for order processing events
resource "aws_sns_topic" "order_processing" {
  name = "${var.service_name}-order-processing-events"

  tags = {
    Name = "${var.service_name}-order-processing-events"
  }
}

# SQS Queue for order processing
resource "aws_sqs_queue" "order_processing" {
  name = "${var.service_name}-order-processing-queue"
  visibility_timeout_seconds = 30    # 30 seconds visibility timeout
  message_retention_seconds = 345600 # 4 days retention
  receive_wait_time_seconds = 20    # 20 seconds long polling
}
```

```

tags = {
    Name = "${var.service_name}-order-processing-queue"
}
}

```

Async API

```

// ProcessOrderAsync handles POST /orders/async - asynchronous order
processing
func (api *OrderAPI) ProcessOrderAsync(c *gin.Context) {

    ***

    // Publish order to SNS topic (non-blocking)
    if api.publisher != nil {
        if err := api.publisher.PublishOrder(&order); err != nil {
            // If publishing fails, mark order as failed
            order.Status = "failed"
            api.store.SetOrder(order.OrderID, &order)
            c.JSON(http.StatusInternalServerError, gin.H{
                "error": "Failed to queue order for processing",
            })
            return
        }
    }

    // Return 202 Accepted immediately (order is queued, not processed
    yet)

    c.JSON(http.StatusAccepted, response)
}

```

```

// processMessage processes a single SQS message
func (op *OrderProcessor) processMessage(workerID int, message
types.Message) {

    // Simulate payment processing
    if err := op.processPayment(order); err != nil {
        log.Printf("Payment failed for order %d: %v", order.OrderID,
err)
    }
}

```



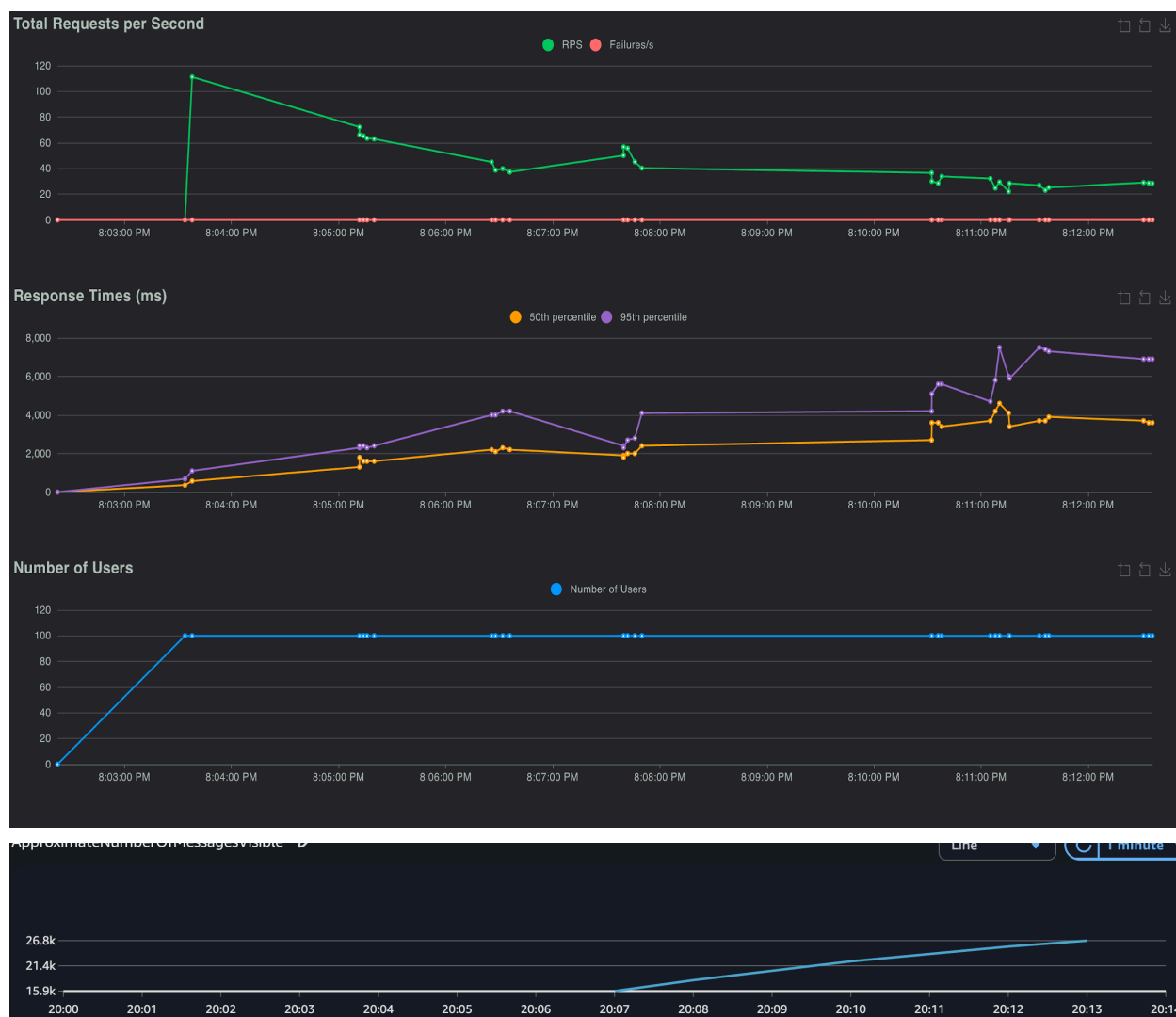
```
        op.deleteMessage(message.ReceiptHandle)
        return
    }
}
```

```
// processPayment simulates payment processing
func (op *OrderProcessor) processPayment(order OrderMessage) error {
    paymentSemaphore <- struct{}{}
    defer func() {
        <-paymentSemaphore
    }()

    time.Sleep(3 * time.Second)

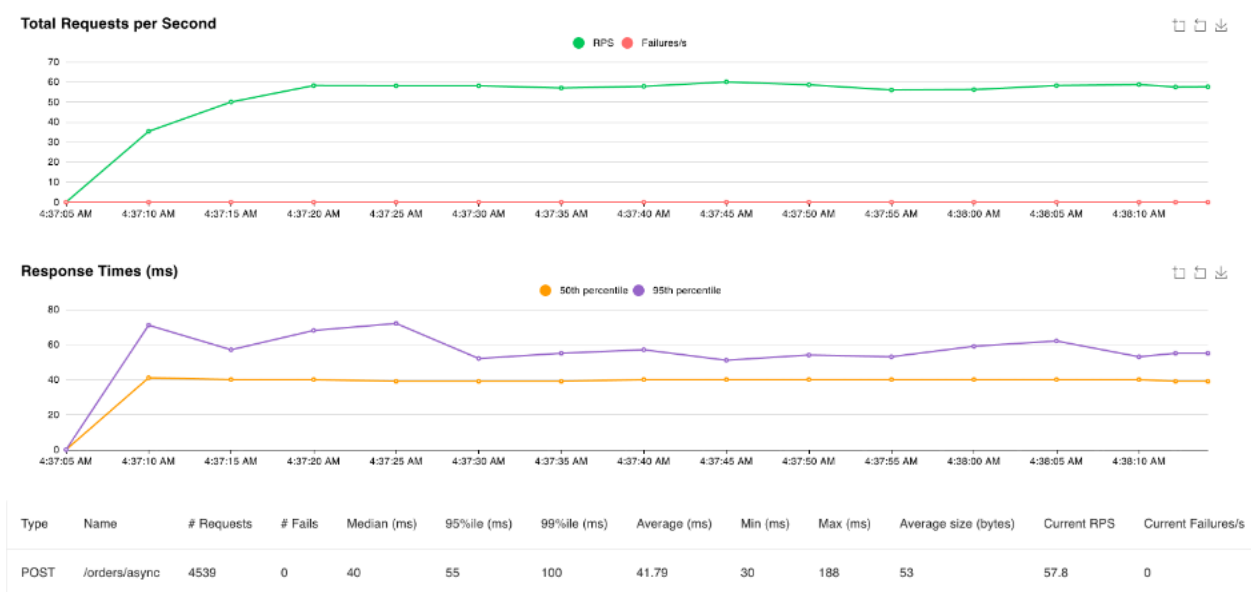
    return nil
}
```

wenshuang

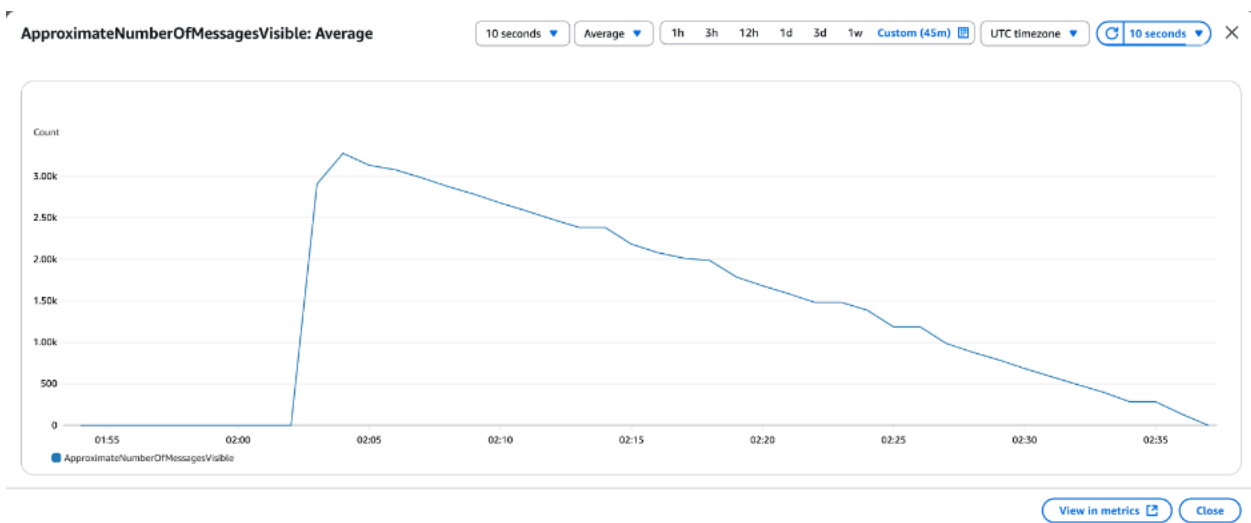


goroutine count	peak depth	growth (message/sec)	time to zero	CPU Utilization (%)
1	28.9k	13	3.7hour	100
5	8.4k	6.65	21min	100
20	7.3k	8.73	18min	100
100	7.4k	8.9	19min	100

Zhixiao:

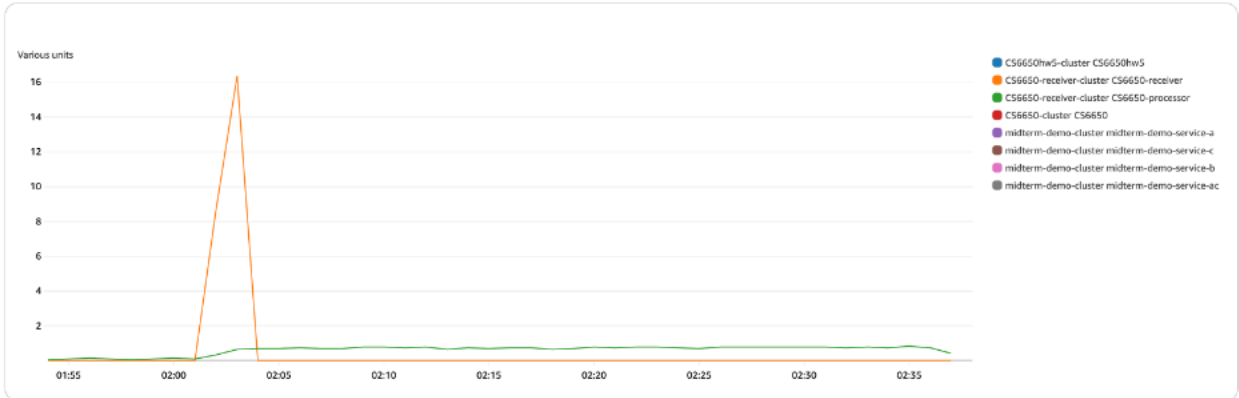


5 go routines:



CPUUtilization: Average

10 seconds ▾ Average ▾ 1h 3h 12h 1d 3d 1w Custom (45m) UTC timezone 10 seconds X

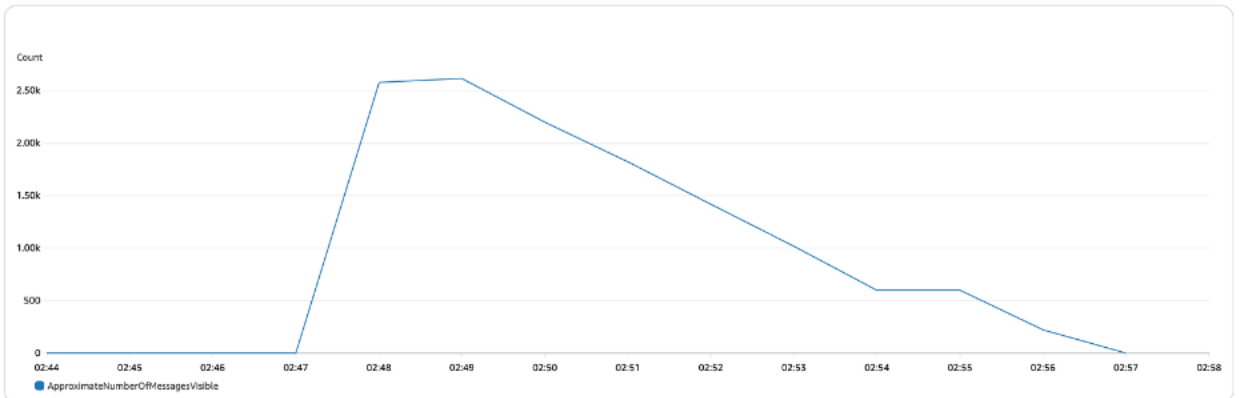


[View in metrics](#) [Close](#)

20 go routines:

ApproximateNumberOfMessagesVisible: Average

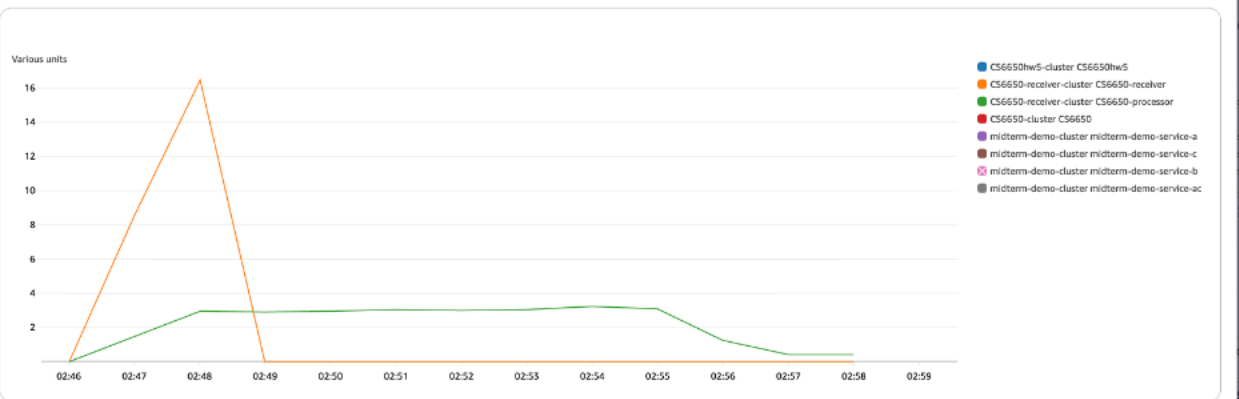
5 seconds ▾ Average ▾ 1h 3h 12h 1d 3d 1w Custom (15m) UTC timezone 10 seconds X



[View in metrics](#) [Close](#)

CPUUtilization: Average

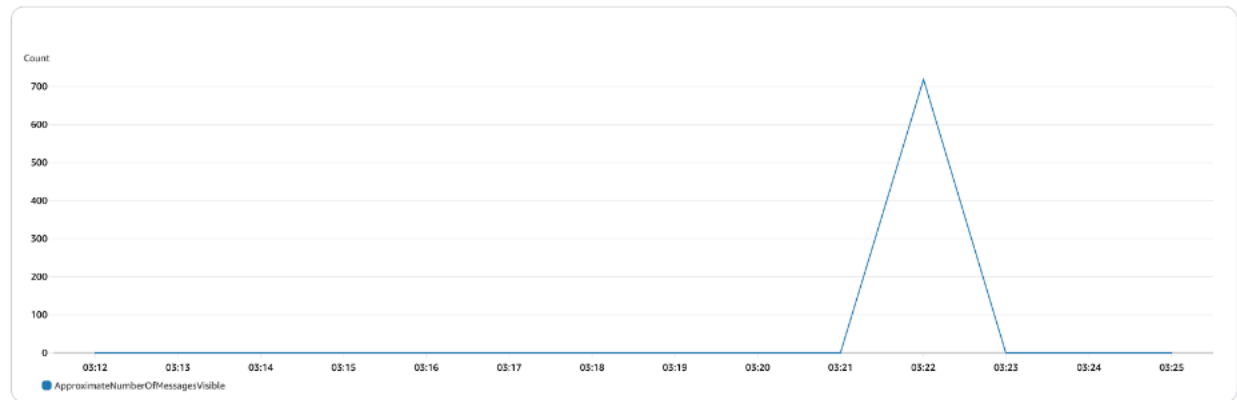
5 seconds ▾ Average ▾ 1h 3h 12h 1d 3d 1w Custom (15m) UTC timezone 10 seconds X



100 go routines:

ApproximateNumberOfMessagesVisible: Average

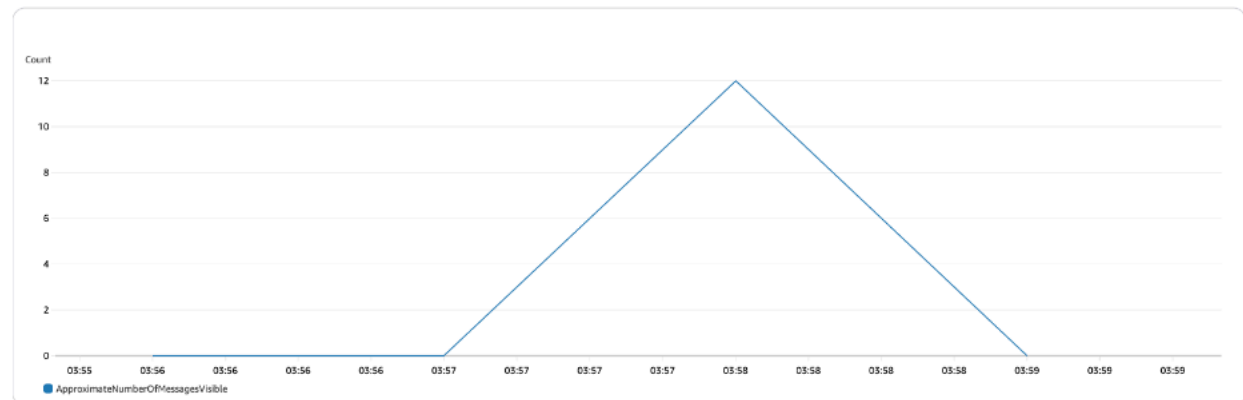
5 seconds ▾ Average ▾ 1h 3h 12h 1d 3d 1w Custom (15m) UTC timezone 10 seconds ▾ X



150 go routines:

ApproximateNumberOfMessagesVisible: Average

5 seconds ▾ Average ▾ 1h 3h 12h 1d 3d 1w Custom (5m) UTC timezone 10 seconds ▾ X

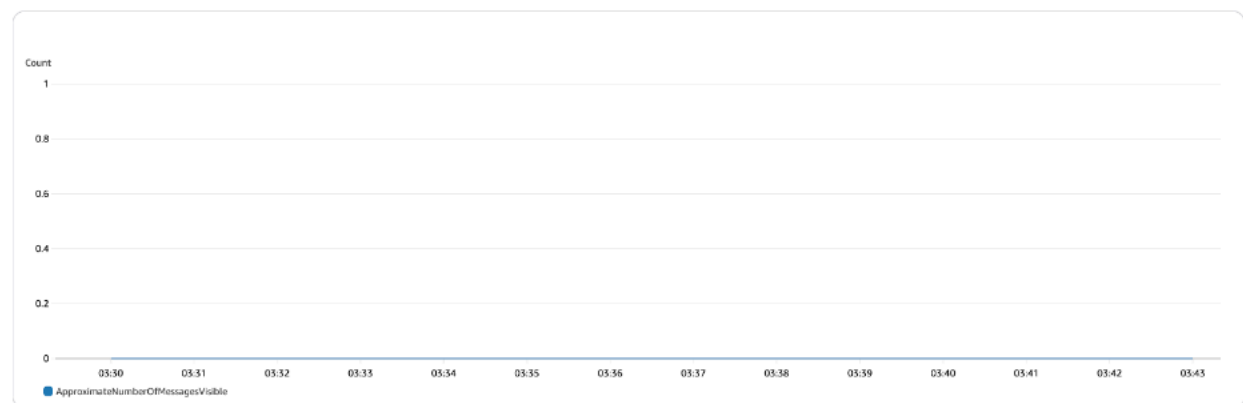


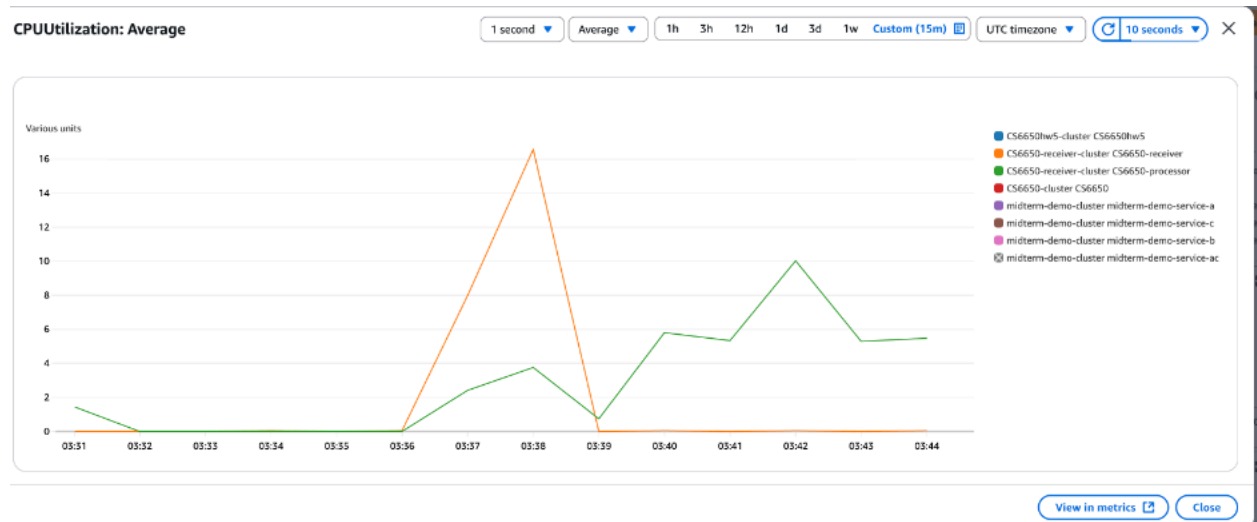
[View in metrics](#) [Close](#)

180 go routines:

ApproximateNumberOfMessagesVisible: Average

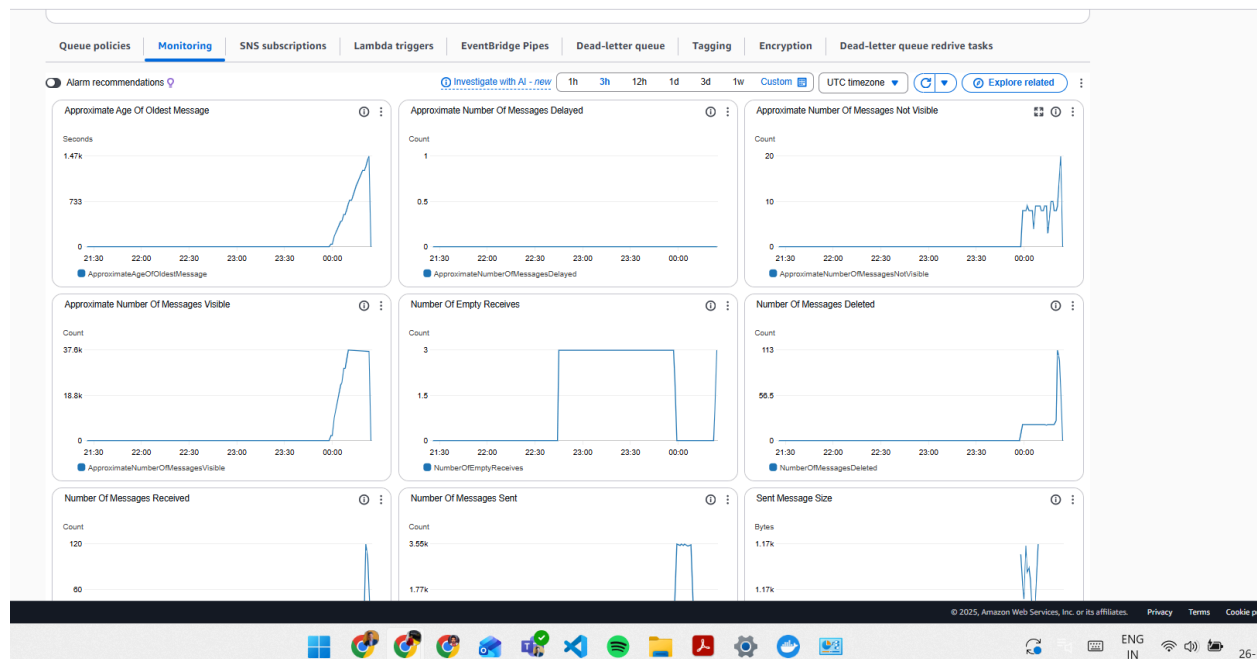
5 seconds ▾ Average ▾ 1h 3h 12h 1d 3d 1w Custom (15m) UTC timezone 10 seconds ▾ X





Goroutine Count	Peak Queue Depth	Time to Zero	Peak Processor CPU
5	~3,000	~35 min	~2
20	~2,500	~12 min	~3
100	~700	~2 min	~15
150	~12	~2 min	TBD
180	~0-1	N/A (real-time)	~10 (could be wrong)

Snahil:



Analysis: Success! With 182 workers, the processing capacity (~60.06 orders/sec) matched the ingestion rate (~58 orders/sec). The SQS chart shows that while messages briefly queued during the 60-second test, the backlog was cleared almost immediately afterward, preventing significant customer delays. The Number Of Messages Deleted chart spiked to ~3.6k, confirming all test messages were processed quickly.

Applying Lambda

```
// handleSNSEvent processes SNS messages triggered by Lambda
func handleSNSEvent(ctx context.Context, snsEvent events.SNSEvent) error {
    for _, record := range snsEvent.Records {
        snsRecord := record.SNS

        // Parse the order message from SNS
        var order OrderMessage
        if err := json.Unmarshal([]byte(snsRecord.Message), &order);
err != nil {
            log.Printf("Error parsing order: %v", err)
            continue
        }
    }
}
```

```

        // Simulate payment processing (3 seconds delay)
        if err := processPayment(order); err != nil {
            log.Printf("Payment failed for order %s: %v",
order.OrderID, err)
            continue
        }
    }

    return nil
}

// processPayment simulates payment processing with 3-second delay
func processPayment(order OrderMessage) error {
    // Simulate payment processing time (3 seconds)
    time.Sleep(3 * time.Second)

    // Silent processing - no logs for production
    return nil
}

func main() {
    lambda.Start(handleSNSEvent)
    fmt.Println("Lambda function started")
}

```

```

# Lambda function
resource "aws_lambda_function" "order_processor" {
    count          = var.enable_lambda ? 1 : 0
    filename       = "${path.module}/../../../../src/lambda/bootstrap.zip"
    function_name  = "${var.service_name}-order-processor-lambda"
    role           = aws_iam_role.lambda_execution.arn
    handler        = "bootstrap"
    runtime        = "provided.al2" # Go runtime
    memory_size    = 512             # 512MB as required
    timeout        = 30              # 30 seconds (needs time for 3-second
processing)

    source_code_hash =
filebase64sha256("${path.module}/../../../../src/lambda/bootstrap.zip")

```



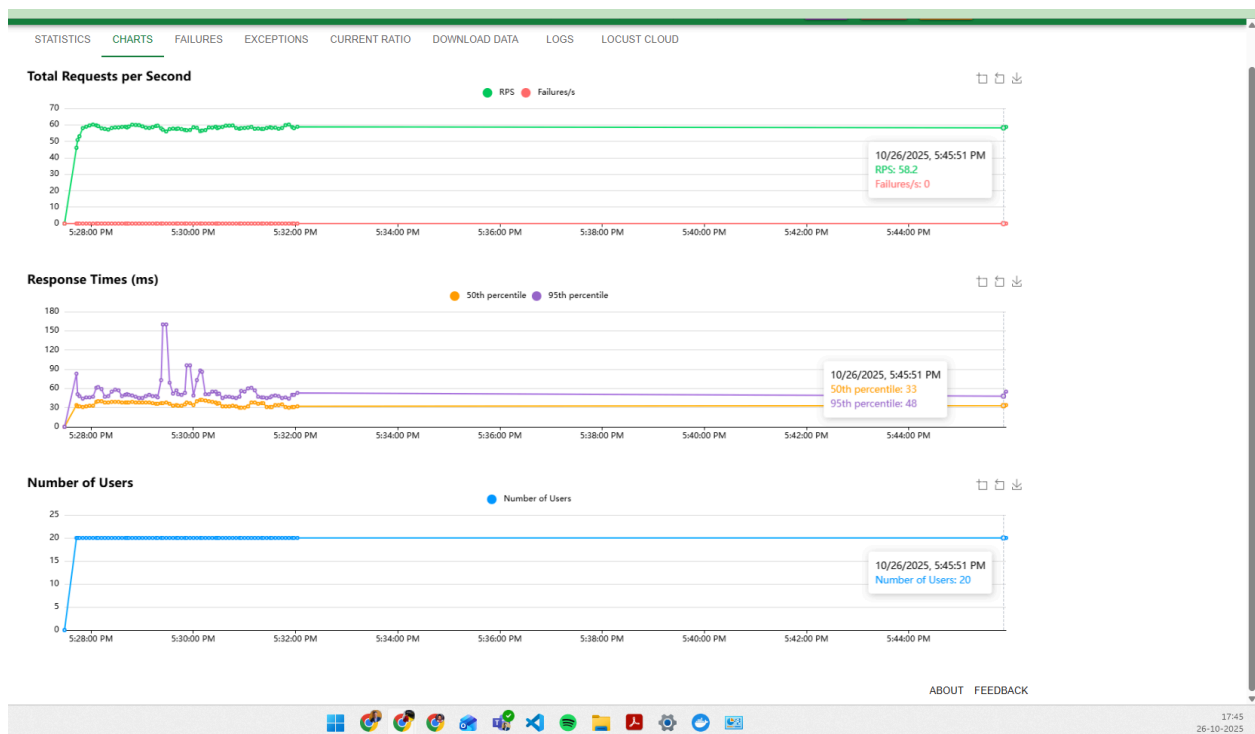
```

environment {
  variables = {
    AWS_REGION = var.region
  }
}

tags = {
  Name = "${var.service_name}-order-processor-lambda"
}
}

```

Snahil after applying lambda:



Lambda Logs

wenshuang

2025-10-23T22:4...	REPORT RequestId: c7b32de9-756a-4223-8d1a-8d3b47f4107b Duration: 3003.47 ms Billed Duration: 3060 ms Memory Size: 512 MB Max Memory U...
	REPORT RequestId: c7b32de9-756a-4223-8d1a-8d3b47f4107b Duration: 3003.47 ms Billed Duration: 3060 ms Memory Size: 512 MB Max Memory Used: 18 MB Init Duration: 56.47 ms
2025-10-23T22:51:39.242-...	REPORT RequestId: 988c64cb-66aa-4e41-b895-86f464e366f0 Duration: 3001.48 ms Billed Duration: 3002 ms Memory Size: 512 MB Max...
	REPORT RequestId: 988c64cb-66aa-4e41-b895-86f464e366f0 Duration: 3001.48 ms Billed Duration: 3002 ms Memory Size: 512 MB Max Memory Used: 18 MB

If it promises to have 10,000 orders/month, using ECS vs. Lambda will cost 17 and 0. Assuming \$5 profit every order will get, 0.034% order lost will lose \$17 a month. 0.034%

is an extremely narrow margin, losing 3-4 orders a month eliminates the cost of ECS. The current system successfully recovered from a 50,000 message backlog with zero message loss, maintained stable processing rates over extended periods, and showed resilience under heavy load conditions. Cold starts, timeout issues, and configuration errors could easily push failure rates above the 0.034% threshold. For a startup where every customer and order matters, the guaranteed processing reliability of ECS provides better user experience than Lambda's cost savings.

Zhixiao:

Cold start:

2025-10-26T07:38:04.973Z	REPORT RequestId: 11cc9917-ec56-4a9f-9183-4c04f96a43ce Duration: 3003.07 ms Billed Duration: 3076 ms Memory Size: 512 MB Max Memory Used: 19 MB Init...
REPORT RequestId: 11cc9917-ec56-4a9f-9183-4c04f96a43ce Duration: 3003.07 ms Billed Duration: 3076 ms Memory Size: 512 MB Max Memory Used: 19 MB Init	Duration: 72.34 ms

Warm start:

2025-10-26T07:56:05.246Z	REPORT RequestId: a80b69c3-4b53-4b7e-9e9b-456e518bde33 Duration: 3001... 2025/10/26/(\$LATEST)194e4a275dbc49d99f653ede373d74e8
REPORT RequestId: a80b69c3-4b53-4b7e-9e9b-456e518bde33 Duration: 3001.45 ms Billed Duration: 3002 ms Memory Size: 512 MB Max Memory Used: 20 MB	

Cold starts cost some overhead time.

Let's assume 1000orders/month:

Requests: 10,000 Cost per million: \$0.20 Calculation: $(10,000 / 1,000,000) \times \$0.20 = \$0.002$ Within free tier (1M requests/month) Request cost: \$0.00

Based on my observations, I recommend switching to Lambda. Cold starts occurred only on first invocations and after ~10-15 minutes of inactivity, adding just 72ms overhead (2.4% of total processing time)—negligible for 3-second payment processing. The cost advantage is compelling: Lambda is FREE for our expected 10,000 orders/month versus \$17/month for ECS. While we lose SQS's durability guarantees (only 2 SNS retries versus unlimited queue-based retries), this is acceptable at our startup scale given Lambda's operational simplicity—zero infrastructure management, automatic scaling, and no queue monitoring. For a small team prioritizing rapid development and cost efficiency, Lambda's benefits outweigh the loss of SQS guarantees.

Snahil:

Analysis

- **Cold Starts:** The observed Init Duration was **407.47 ms**. This adds ~13.6% overhead to the first 3-second processing task but does not affect subsequent warm invocations (which took ~3005 ms).
- **Cost Comparison:**
 - ECS (2 tasks): ~\$17/month (fixed).
 - Lambda: \$0/month (within the free tier of 1M requests + 400k GB-seconds, covering ~267k orders/month for this workload).
- **Trade-offs:**
 - **Gained:** Zero operational overhead (no queue/worker management, auto-scaling), pay-per-use pricing (effectively free for low volume).
 - **Lost:** SQS buffering/retry guarantees (SNS retries twice then discards), SQS batching capabilities, and potential minor cold start latency.

Recommendation

Switch Recommendation: Yes, the startup should switch to Lambda.

Justification: For a startup, the Lambda architecture offers significant advantages. The **\$0 cost** within the generous free tier and the **elimination of operational overhead** (no need to manage ECS scaling, monitor queues, patch servers) are compelling. The development team can focus on features, not infrastructure management. The observed 407ms cold start is acceptable for this background task. While the lack of SQS guarantees is a risk, it can be mitigated with a Dead Letter Queue (DLQ) for failed SNS messages and robust logging within the Lambda function. The benefits of cost savings and simplified operations heavily outweigh the manageable risks for this specific workload and startup context.

Sacrificing the robustness of SQS queuing for dramatic cost savings and the complete elimination of operational overhead. For the target startup scenario, the Lambda architecture proved to be the superior choice due to its

cost-effectiveness and operational simplicity.